

passgen

Детерминированный генератор паролей

Технологии: Python + HTML/CSS/JS

HMAC-SHA256, Web Crypto API

Кроссплатформенно: Windows / Linux

1. Идея проекта

Проблема: человек не может запомнить сложные пароли для каждого сайта. Менеджеры паролей хранят их в базе — если взломают, утекут все.

Решение: пароль не хранится, а вычисляется каждый раз заново на основе одной секретной фразы (seed) и названия сервиса.

Seed + имя сайта = всегда один и тот же пароль. Не нужно ничего хранить — только помнить одну фразу.

Как использовать

- Ввести сид (секретную фразу)
- Выбрать/ввести название сервиса
- Настроить длину и набор символов
- Нажать "Создать" — пароль готов

2. Архитектура

index.html — всё в одном файле

- HTML — разметка карточек (.card)
- CSS — переменные в :root, анимации
- JS — Web Crypto API, HMAC-SHA256, логика

app.py — HTTP-сервер

- Python, ноль зависимостей
- Открывает index.html в браузере (без адресной строки)
- Порт 8765, если занят — следующий

Основные функции JS

init() — загрузка и инициализация

gen() — получает данные, запускает HMAC

hmacGen() — сама HMAC-генерация

genLogin() — генерация логина

Сохранение настроек — localStorage

3. Алгоритм HMAC-SHA256

Входные данные

- seed — секретная фраза (ключ)
- service — название сервиса (сообщение)
- length — желаемая длина
- alphabet — набор разрешённых символов

Процесс

1. Seed используется как HMAC-ключ через Web Crypto API
2. Сообщение = service + length + счётчик
3. HMAC возвращает 32 байта
4. Каждый байт превращается в символ: $\text{byte} \% \text{len}(\text{alphabet})$
5. Байты $\geq 256 - (256 \% \text{len})$ отбрасываются (rejection sampling)
6. Если нужно больше 32 символов — счётчик ++, новый хэш

Результат: одинаковые входные данные = всегда одинаковый пароль. Seed никуда не передаётся — всё в браузере.

4. Интерфейс

Карточки (сверху вниз)

- Доступ: поле для сида
- Сервис: ввод + список с логотипами
- Параметры: пресеты, длина, наборы, расширенные
- Логин и пароль: вывод + полосы надёжности

Оценка надёжности

- Розовый — слабый (< 10 симв. или < 3 наборов)
- Малиновый — средний (≥ 10 симв. и 3 набора)
- Сиреневый — сильный (≥ 14 симв. и 4 набора)

Инверсия

- Первая заглавная — `toUpperCase()`
- Последняя цифра — через `HMAC(seed+lastdig)`

5. Технологии и запуск

Что используется

- Python 3 — встроенный http.server
- Web Crypto API — HMAC-SHA256 в браузере
- localStorage — хранение настроек
- Ноль зависимостей — npm/pip не нужны
- Один файл index.html — HTML+CSS+JS

Запуск

Linux:

```
python3 app.py
```

Windows:

```
python app.py
```

Без установки. Просто клонировать и запустить.

Преимущества

- Пароли не хранятся — вычисляются
- Seed никуда не передаётся — безопасно
- Кроссплатформенно
- Экспорт/импорт настроек (без seed)
- Автокопирование
- Генерация логина